2026-7-12 Linux进程--状态

- 1. 怎样理解进程状态
  - 1.1 状态是进程当前能不能继续运行
    - 1.1.1 状态保存在进程管理结构中
    - 1.1.2 状态也可以从队列位置理解
  - 1.2 操作系统教材中的通用状态
    - 1.2.1 新建、就绪、运行和结束
    - 1.2.2 就绪不等于正在 CPU 上运行
    - 1.2.3 阻塞是等待某个事件
    - 1.2.4 挂起与内存交换有关
    - 1.2.5 阻塞与挂起不是同一件事
- 2. Linux 的进程状态
  - 2.1 Linux 用状态字母表示进程情况
    - 2.1.1 七种常见状态
    - 2.1.2 R：运行或就绪
    - 2.1.3 S：可中断睡眠
    - 2.1.4 D：不可中断睡眠
    - 2.1.5 T 与 t：暂停状态
    - 2.1.6 X：死亡状态
    - 2.1.7 Z：僵尸状态
  - 2.2 如何查看进程状态
    - 2.2.1 使用 ps 查看 STAT
    - 2.2.2 STAT 后面的加号表示前台进程组
- 3. 僵尸进程
  - 3.1 子进程退出后为什么要保留退出信息
    - 3.1.1 父进程需要知道子进程怎样结束
    - 3.1.2 Z 状态只保留最小必要信息
  - 3.2 父进程如何回收子进程
    - 3.2.1 wait 与 waitpid
    - 3.2.2 不回收会怎样
    - 3.2.3 僵尸进程不占 CPU，但会占进程管理资源
  - 3.3 用户态内存泄漏与僵尸进程的区别
    - 3.3.1 进程结束后用户态内存会被回收
    - 3.3.2 僵尸进程泄漏的是内核管理资源
- 4. 孤儿进程
  - 4.1 父进程先退出

        - 4.1.1 孤儿进程仍然活着
    - 4.2 谁来接管孤儿进程
        - 4.2.1 init 或 systemd 成为新的收养者
        - 4.2.2 为什么需要统一收养
- 5. 进程状态的统一理解
    - 5.1 从队列和转换关系理解状态
    - 5.2 自检问题

> **本篇范围**
>
> 前置阅读：2026-7-7 Linux进程--基础概念、2026-7-12 Linux进程--理解。本篇只整理进程状态的表示、状态转换、Linux 七种状态、僵尸进程与孤儿进程；进程优先级、完整调度算法和切换流程留到后续专题。

1. 怎样理解进程状态

1.1 状态是进程当前能不能继续运行

1.1.1 状态保存在进程管理结构中

进程不仅是一份代码，还处在某个具体执行阶段。它可能正在使用 CPU，也可能在等待键盘输入、磁盘 I/O 或锁；可能因为内存不足被暂时换出，也可能已经退出但等待父进程读取退出结果。

这些阶段会被记录在进程的管理信息中，Linux 对应 `task_struct` 的状态字段。因此，查看进程状态，本质上是在查看内核为这个进程维护的当前情况。

进程状态至少要回答两个问题：

1. 这个进程现在有没有条件继续执行？
2. 如果没有，它在等待什么，或者被谁暂时移出了运行条件？

1.1.2 状态也可以从队列位置理解

操作系统管理进程采用“先描述、再组织”的方法：

- 用 `task_struct` 描述进程；
- 把 `task_struct` 放入运行队列、等待队列等组织结构中。

所以从内核角度看，状态的差异常常表现为：

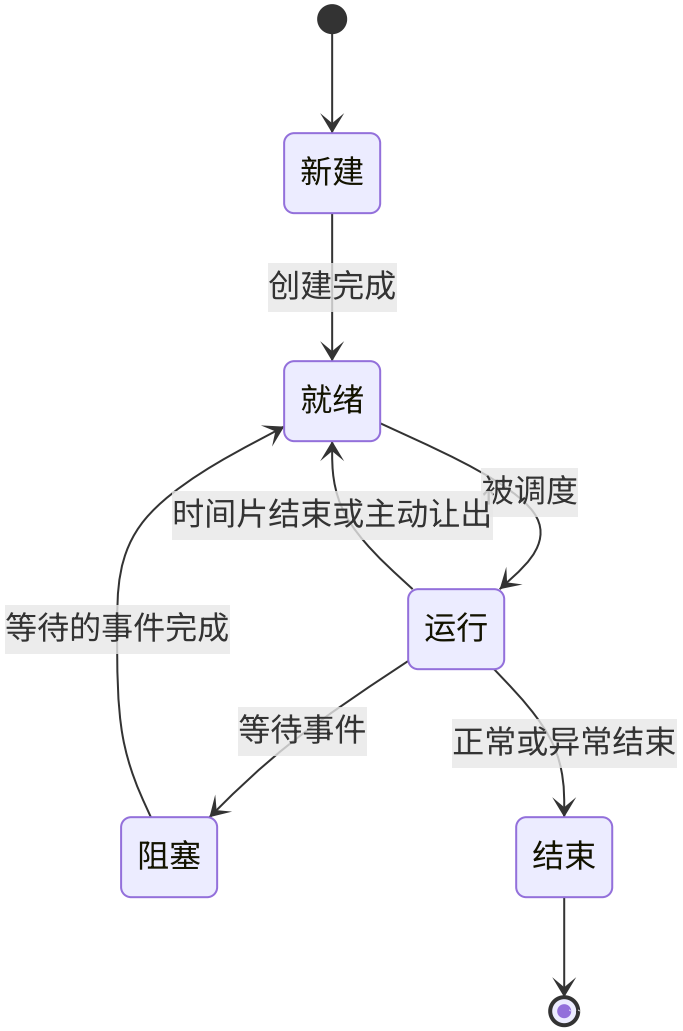
> **进程的 `task_struct` 当前在谁的队列里，或者是否正由 CPU 执行。**

运行队列中的进程有机会获得 CPU；等待队列中的进程通常要等某个事件发生。不同事件会对应不同的等待队列，例如等待键盘输入、等待磁盘 I/O、等待锁释放等。

1.2 操作系统教材中的通用状态

1.2.1 新建、就绪、运行和结束

教材通常把进程状态简化为：**运行、阻塞、挂起**。如果把运行阶段继续细分，可以得到以下基本转换：



这里的状态变化不是由进程自己随意切换的，而是由进程发出的请求和操作系统调度共同决定。

1.2.2 就绪不等于正在 CPU 上运行

就绪表示进程已经具备运行条件，只差 CPU；**运行**表示它正在某个 CPU 核心上执行。

在多进程系统中，CPU 数量可能少于就绪进程数量，因此很多进程可能都“准备好运行”，但同一时刻只有一个进程能使用某个 CPU 核心。

Linux 没有把这两种情况拆成两个不同字母，而是都用 `R` 表示，这是后文容易混淆的一点。

1.2.3 阻塞是等待某个事件

阻塞表示进程当前不能继续执行，必须等待某个条件成立。例如：

- 等待键盘或网卡数据；
- 等待磁盘读写完成；
- 等待锁被其他进程释放；
- 等待其他进程提供资源。

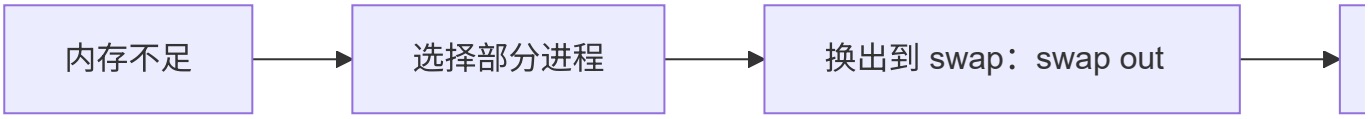
进程虽然不再获得 CPU，但仍然存在，它的 `task_struct` 会进入相应事件的等待队列。事件完成后，操作系统会把它从等待状态转回就绪状态。

硬件就绪后通常由操作系统最先获知，因为设备驱动和中断处理都由内核管理。内核再根据等待队列唤醒相应进程，这也体现了操作系统“管理硬件、再通过数据结构组织进程”的职责。

理解阻塞的关键不是看程序“看起来有没有在运行”，而是看它是否缺少继续执行所需的条件。

1.2.4 挂起与内存交换有关

挂起主要从内存资源角度理解。当系统内存不足时，操作系统可以把某些进程的代码或数据从内存换到磁盘上的 swap 分区，从而腾出内存空间。



- **换出（swap out）**：把部分进程内容从内存移到 swap；
- **换入（swap in）**：将来需要执行时，再把内容从 swap 移回内存。

swap 的本质是**用时间换空间**。访问 swap 需要磁盘 I/O，速度远低于访问内存，因此过度使用 swap 会让系统明显变慢。被换出的进程不等于已经结束，它只是暂时不具备完整驻留内存的条件。

swap 通常是单独的分区或交换文件，不作为普通目录供用户直接浏览。它适合作为内存不足时的缓冲，但不宜配置得过大，否则系统可能长期依赖低速磁盘来维持运行，掩盖实际的物理内存压力。

如果 swap 也已经无法缓解内存压力，Linux 可能选择终止部分进程来保护系统。

1.2.5 阻塞与挂起不是同一件事

对比项	阻塞	挂起
根本原因	等事件或资源	内存不足，部分内容被换出
进程是否还存在	存在	存在
是否一定等待外部事件	是	不一定
可否同时发生	可以形成“阻塞挂起”	可以形成“阻塞挂起”

一个进程可以因为等待磁盘 I/O 而阻塞；内存不足时，它还可能进一步被换出，成为阻塞挂起状态。

2. Linux 的进程状态

2.1 Linux 用状态字母表示进程情况

2.1.1 七种常见状态

在 Linux 中通过 `ps` 等工具查看进程时，常见状态字母如下：

状态	含义	基本理解
`R`	Running	正在运行或已经就绪，等待 CPU
`S`	Interruptible Sleep	可中断睡眠，通常等待事件

状态	含义	基本理解
`D`	Uninterruptible Sleep	不可中断睡眠，常见于等待 I/O
`T`	Stopped	被信号暂停
`t`	Tracing Stop	被调试器追踪而暂停
`X`	Dead	死亡状态，通常是短暂过渡状态
`Z`	Zombie	已退出，但退出信息尚未被父进程回收

内核内部用位标志表示这些状态，用户工具再将其显示为字母。状态值是内核调度和进程管理的重要依据。

2.1.2 R：运行或就绪

`R` 并不代表进程一定正在 CPU 上计算。Linux 同时把以下两种情况归入 `R`：

- 进程正在 CPU 上执行；
- 进程已经具备运行条件，正在运行队列中等待 CPU。

所以把 `R` 理解为“运行或就绪”更准确。一个 `R` 状态进程是否真正占用 CPU，需要看它是否被调度到。

2.1.3 S：可中断睡眠

`S` 是最常见的等待状态，表示进程正在可中断睡眠，通常是在等待某个事件完成。

例如：

```c
#include <stdio.h>

int main(void)
{
    int value;
    scanf("%d", &value);
    printf("value = %d\n", value);
    return 0;
}
```

程序阻塞在 `scanf()` 时，通常会以 `S` 状态等待键盘输入。键盘事件完成后，内核唤醒它，它转入可运行状态。

“可中断”意味着进程在等待过程中可以被信号唤醒或终止。例如：

```
kill -KILL <PID>
```

`SIGKILL` 是不可被忽略的强制终止信号。课堂手记中也写作 `kill -9 <PID>`，其中 `9` 是 `SIGKILL` 的信号编号。

2.1.4 D：不可中断睡眠

`D` 表示不可中断睡眠，常见于等待磁盘 I/O 等关键内核操作。这个状态通常不能被普通信号打断，进程必须等待当前内核操作完成，或者等待相关条件解除。

可以把它理解为：进程正在等待一件不容易被“中途取消”的事情完成。大量 I/O 操作时可能观察到 `D` 状态。

因此，发送 `SIGKILL` 后进程也不一定立即消失：如果它正处在不可中断等待中，信号要等到进程有机会处理时才会生效。不要看到 `D` 就立即判断系统卡死，还应结合 I/O、挂载存储和设备状态分析。

2.1.5 T 与 t：暂停状态

`T` 表示进程被显式暂停，通常由信号触发。可以使用：

```
kill -l
kill -STOP <PID>
kill -CONT <PID>
```

课堂手记中的 `kill -19` 和 `kill -18` 分别对应常见的 `SIGSTOP`、`SIGCONT` 写法。实际使用中优先写信号名称更清楚：

- `SIGSTOP`：暂停进程；
- `SIGCONT`：让暂停的进程继续运行；
- `SIGKILL`：终止进程。

`t` 表示追踪暂停，常见于 `gdb` 调试。调试器控制被调试进程，使其在断点或单步执行时暂停，以便观察执行过程。

`T` 和 `t` 都是暂停，但触发场景不同：前者通常是普通的停止信号，后者与调试追踪有关。

2.1.6 X：死亡状态

`X` 是进程已经死亡的短促状态。这个状态通常是内核完成退出处理的过渡阶段，不会像 `Z` 那样长期停留在普通任务列表中。

进程真正结束并完成回收后，内核释放它的管理结构和 PID，用户再通过普通进程列表通常看不到这个进程。

2.1.7 Z：僵尸状态

`Z` 表示子进程已经退出，但仍保留最小限度的进程管理信息，等待父进程读取退出结果。

僵尸进程不能继续执行代码，也不能被普通信号“救活”。它保留 `task_struct` 的目的，是让父进程知道：

- 子进程是否正常结束；
- `main()` 返回了什么；
- 是否因某个信号终止。

具体的形成和回收过程见后文。

2.2 如何查看进程状态

2.2.1 使用 ps 查看 STAT

`ps` 的 `STAT` 列显示进程状态。可以先显示标题，再筛选目标进程：

```
ps axj | head -n 1
ps axj | grep myprocess
```

也可以指定 PID：

```
ps -p <PID> -o pid,ppid,stat,cmd
```

观察时不要只看状态字母，还应结合 PID、PPID、命令名和当前 I/O 情况判断进程在做什么。

2.2.2 STAT 后面的加号表示前台进程组

`ps` 中可能看到 `S+`、`R+` 等状态。末尾的 `+` 表示该进程属于当前终端的前台进程组。

一般可以这样区分：

- 带 `+`：前台进程，通常可以接收终端输入；
- 不带 `+`：后台进程或没有当前终端前台关系的进程。

使用 `&` 可以使命令在 shell 的后台运行：

```
./myprocess &
```

后台进程仍然可以运行，但它通常不能像前台进程一样直接从当前终端读取输入。

3. 僵尸进程

3.1 子进程退出后为什么要保留退出信息

3.1.1 父进程需要知道子进程怎样结束

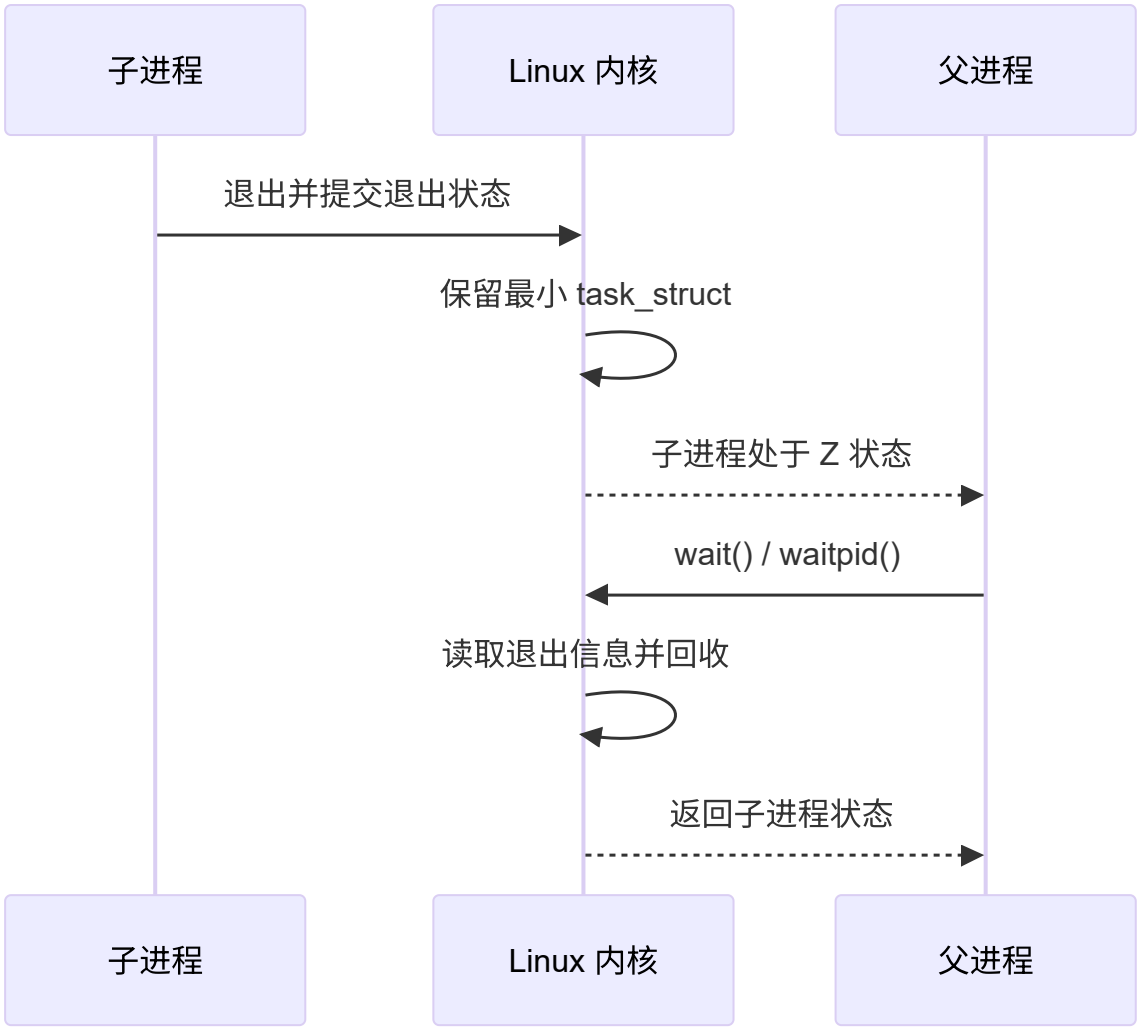
父进程创建子进程，往往是要让它完成某项任务。子进程退出时，父进程需要知道任务结果：

- 是否正常返回；
- 退出码是什么；
- 是否被信号终止；
- 被哪个信号终止。

因此，子进程不能退出后立刻把所有痕迹清除，否则父进程就失去了获取结果的依据。

3.1.2 Z 状态只保留最小必要信息

子进程正常退出后，大部分用户态资源会被释放，但内核仍保留其 `task_struct` 中的退出状态等必要信息，使它暂时处于 `Z` 状态。



所以僵尸进程不是“还在偷偷运行”的进程。它已经死亡，只剩下供父进程读取的退出信息。

3.2 父进程如何回收子进程

3.2.1 wait 与 waitpid

父进程通常通过系统调用回收子进程：

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

- `wait()` 等待并回收一个子进程；
- `waitpid()` 可以指定要等待的子进程及相关选项。

回收时，父进程取得子进程的退出状态，内核随后释放对应的 `task_struct`。此时僵尸进程才真正消失。

3.2.2 不回收会怎样

如果子进程已经退出，父进程仍在运行，却一直不调用 `wait()` 或 `waitpid()`，子进程就会长期保持 `Z` 状态。

每个僵尸进程仍然保留一份内核管理结构。父进程不断创建子进程却不回收时，这些结构会持续占用内核资源，形成**资源泄漏**。这不是普通用户态的 `malloc` 泄漏，但危害同样真实。

3.2.3 僵尸进程不占 CPU，但会占进程管理资源

僵尸进程已经不能运行，因此不会继续消耗 CPU 执行时间。它占用的主要是进程管理信息和退出状态。

它也不能通过 `kill -9` 直接清除，因为它已经退出，问题不在终止，而在父进程没有读取退出结果、内核没有完成回收。

正确的处理方向是：

1. 找出僵尸进程的父进程；
2. 让父进程调用 `wait()` / `waitpid()`；
3. 或者终止父进程，让僵尸进程被新的收养者回收。

3.3 用户态内存泄漏与僵尸进程的区别

3.3.1 进程结束后用户态内存会被回收

普通进程结束后，操作系统会回收它拥有的用户态地址空间和大部分资源。因此，一个短命程序即使中途通过 `malloc` / `new` 申请内存后没有手动释放，进程结束后通常也不会继续保留这些内存。

> ✎ **常驻进程要单独考虑**

服务器、守护进程等会长期运行。只要进程不结束，反复申请却不释放的内存就会持续累积，最终可能导致 OOM。因此，“进程结束会自动回收”不能成为长期运行程序忽略内存管理的理由。

3.3.2 僵尸进程泄漏的是内核管理资源

普通内存泄漏通常发生在进程仍然存活期间；僵尸进程泄漏发生在进程已经死亡之后。

对比项	用户态内存泄漏	僵尸进程
存活情况	进程通常仍活着	子进程已经退出
主要占用	用户态内存	`task_struct` 等内核管理资源
是否耗 CPU	可能	通常不消耗
解决方向	修正申请与释放逻辑	父进程回收子进程

4. 孤儿进程

4.1 父进程先退出

4.1.1 孤儿进程仍然活着

如果父进程先于子进程结束，子进程仍在运行，它就成为**孤儿进程**。此时它的亲生父进程已经不存在，但这并不意味着子进程会自动退出。

孤儿进程与僵尸进程的主语恰好相反：

类型	谁先退出	当时状况
僵尸进程	子进程先退出	子进程已死，等待父进程读取结果
孤儿进程	父进程先退出	子进程仍在运行，失去原父进程

因此，“孤儿进程”强调失去父进程，“僵尸进程”强调退出信息未回收。

4.2 谁来接管孤儿进程

4.2.1 init 或 systemd 成为新的收养者

Linux 通常会把孤儿进程收养到 PID 1 的 `init` 或 `systemd` 进程名下。等孤儿进程将来退出时，由这个新的父进程负责回收，避免它长期成为僵尸进程。

在容器等特殊运行环境中，回收职责可能由容器内的 PID 1 进程或专门的 init 程序承担。

4.2.2 为什么需要统一收养

如果没有进程接管孤儿进程，它退出后可能没有可靠的父进程读取退出状态，最终留下僵尸进程。

由一个长期存在、负责系统初始化和子进程回收的 PID 1 进程接管，可以维持进程树的完整性和回收链路。

5. 进程状态的统一理解

5.1 从队列和转换关系理解状态

进程状态不是孤立的标签，而是一条连续的转换链：

```
创建
  ↓
就绪 / Linux R
  ↓ 被调度
运行 / Linux R
  ├─ 时间片结束 → 就绪
  ├─ 等待事件   → 阻塞 S / D
  ├─ 收到停止信号 → T / t
  └─ 结束       → X
                  └─ 父进程尚未回收 → Z
```

内存不足时，就绪或阻塞进程还可能被换出到 swap，形成挂起状态；条件恢复后再被激活或换入。

可以用两句话概括：

> **阻塞看事件，挂起看内存，运行看是否具备 CPU 执行条件。**
>
> **僵尸看退出信息是否被回收，孤儿看父进程是否先退出。**

5.2 自检问题

应该能够区分这些情况

1. 为什么 Linux 的 `R` 既可能是正在运行，也可能是在等待 CPU?
2. `S` 和 `D` 都在等待，为什么可中断性不同?
3. 阻塞和挂起分别由什么原因造成?
4. 僵尸进程为什么不消耗 CPU，却仍然会造成资源泄漏?
5. 为什么对僵尸进程执行 `kill -9` 没有作用?
6. 孤儿进程为什么不一定处于僵尸状态?
7. `wait()` / `waitpid()` 最终释放的是什么?

后续专题

进程优先级、运行队列的选择策略和完整切换流程见 2026-7-12 Linux进程--切换和调度。